Introdução ao Backend

& Desenvolvimento Orientado do Projeto

Roteiro da Aula Equilibra Fundamentos Teóricos e Prática

Duração total de 4 horas divididas em dois blocos complementares de 2 horas

1ª e 2ª Horas: Fundamentos

Diferenciação conceitual clara de Front-end e Back-end

A dinâmica das requisições e respostas na arquitetura Cliente/Servidor

O papel das regras de negócio executadas de forma segura

O que são APIs, Endpoints e o protocolo HTTP (GET, POST, PUT, DELETE)

JSON como o formato universal de representação de dados

Visão integrada: do navegador ao banco de dados

3ª e 4ª Horas: Prática Orientada

Foco no desenvolvimento orientado do projeto atual

Trabalho focado em HTML, CSS e interações dinâmicas em JavaScript

Melhorias de usabilidade, organização visual e correção de erros

Nova Exigência: identificação de dados fixos que deverão vir da API

Marcando os pontos de integração no código JavaScript existente

Preparação do ambiente e gancho técnico para as implementações em .NET

PARTE 1

Fundamentos Conceituais de Backend

Compreendendo o fluxo de dados, APIs, endpoints e requisições

O Front-End Exibe a Interface, o Back-End Processa os Bastidores

A divisão de responsabilidades garante segurança, performance e organização das regras de negócio

Aspecto	FRONT-END (O que o usuário vê)	BACK-END (O que acontece por trás)
Tecnologias	HTML (estrutura), CSS (aparência), JavaScript (comportamento e interações)	C# / .NET, frameworks de servidor, Drivers de banco de dados, Middlewares
Foco Principal	Interface do usuário, navegação, gráficos, formulários de entrada e feedbacks de tela	Processamento pesado, lógica central, autenticação, controle de acessos e consistência
Armazenamento	Local temporário (LocalStorage, Cookies), sem persistência robusta e segura	Banco de dados relacional ou NoSQL, persistência física, backup e integridade
Regras do Sistema	Aplicações de validação visual simples (ex: se campo de e-mail contém '@')	Verificações profundas (ex: autenticar se a senha do login confere com o banco de dados)

Pergunta para refletir em sala de aula:

“Se eu fizer um formulário de login em HTML, CSS e JavaScript, onde o sistema verifica se a senha está correta?”

Navegadores Atuam como Clientes que Solicitam Recursos ao Servidor

O ciclo estruturado de Requisição (Request) e Resposta (Response) rege as interações na web

CLIENTE (Navegador)

Responsável por iniciar o contato. O front-end HTML/JS formata uma solicitação estruturada.

Exemplo: Usuário clica para carregar o dashboard de vendas.



REQUISIÇÃO (HTTP)

O canal de comunicação que leva o pedido. Contém o endereço (URL), método HTTP e dados necessários.

Exemplo: Solicitação enviada via rede pelo protocolo.



SERVIDOR (Backend)

Processa a lógica, autentica o acesso, interage com o banco de dados e monta o resultado.

Exemplo: Retorna um pacote de dados com vendas e faturamento.

Exemplo do Ciclo na Prática:

O usuário abre uma tela de faturamento -> O JavaScript (cliente) dispara uma requisição de busca -> O backend (servidor) recebe o chamado, busca o faturamento consolidado e responde com os dados organizados -> O front-end renderiza os gráficos e valores na tela do usuário.

Regras de Negócio Importantes Devem Ser Executadas no Servidor

A lógica de processamento garante a integridade das regras de negócio e protege dados críticos

Cadastrar e consultar clientes de forma segura.

Registrar vendas validando integridade financeira.

Calcular descontos com base em privilégios (ex: VIP).

Efetuar autenticação de login de maneira criptografada.

Controlar níveis de acesso e permissões de usuários.

Acessar e registrar dados no banco de dados corporativo.

Lógica Crítica: As regras importantes nunca devem depender do HTML ou da interface. Elas pertencem ao backend.

Fluxo de Regra de Negócio Segura

1. Pedido de Compra
O cliente envia o carrinho com valor de R$ 700

2. Verificação de Perfil
Backend consulta se o cliente possui status VIP

3. Aplicação da Regra
O sistema valida: VIP = SIM -> Aplica 10% de desconto

4. Conclusão Segura
Backend recalcula: Total = R$ 630 e grava a venda

APIs Organizam a Conversa Entre o Front-End e o Back-End

A API age como um mensageiro estruturado que abstrai a complexidade operational da aplicação

Analogia Clássica: O Restaurante

CLIENTE (Mesa): Faz o pedido do menu. Não entra na cozinha nem manuseia os fogões.

GARÇOM (Mensageiro): Anota o pedido estruturado, leva até a cozinha e traz o prato pronto de volta.

COZINHA (Bastidores): Processa os ingredientes, executa a receita e finaliza a entrega.

Mesa (Cliente) → Garçom → Cozinha

Na Prática: A Aplicação Web

FRONT-END (Interface): Solicita uma informação ou operação (ex: listar alunos) de forma simples.

API (Mensageiro / Gateway): Recebe as solicitações do front-end por canais e regras pré-definidas.

BACKEND (Servidor/BD): Processa as regras complexas, manipula dados físicos e envia a resposta de volta.

Front-End → API → Back-End

Abstração Importante: O front-end não precisa conhecer o banco ou o código interno. Ele só precisa saber chamar a API.

Endpoints e Verbos HTTP Mapeiam Recursos e Operações

Uma API disponibiliza endereços específicos para que o cliente acesse os recursos do sistema

Endereços de API (Endpoints)

Endpoints são rotas específicas associadas a recursos da aplicação.

Exemplos de rotas padrão:

- /api/alunos
- /api/produtos
- /api/vendas
- /api/clientes

A mesma rota pode se comportar de formas diferentes dependendo do verbo HTTP usado na requisição.

Métodos HTTP e Relação com o CRUD

Verbo	CRUD	Ação Prática	Exemplo de Rota
GET	READ	Buscar informações	GET /api/alunos (Lista alunos)
POST	CREATE	Inserir novo registro	POST /api/alunos (Cadastra aluno)
PUT	UPDATE	Alterar registro existente	PUT /api/alunos (Edita dados)
DELETE	DELETE	Excluir um registro	DELETE /api/alunos (Remove aluno)

JSON Padroniza a Troca de Dados Estruturados Entre Sistemas

O formato leve e baseado em texto facilita a leitura humana e o processamento em qualquer linguagem

Exemplo de Objeto JSON Único

```
{
  "nome": "Maria",
  "curso": "Ciência de Dados",
  "semestre": 6
}
```

Exemplo de Lista (Array) JSON

```
[
  { "nome": "Maria", "nota": 8.5 },
  { "nome": "João", "nota": 7.2 }
]

// Fluxo típico:
Backend → JSON → API → JS → HTML
```

Por que usamos JSON?

Ele representa a ponte perfeita. O backend estruturado em C# traduz seus objetos para strings JSON. O JavaScript no navegador entende nativamente esse formato e atualiza a interface dinamicamente.

A Arquitetura Completa de Dados Conecta a Tela ao Banco de Dados

Entenda o fluxo ponta a ponta que ocorre quando um usuário faz uma ação na interface

1. Usuário

O usuário realiza uma ação de visualização na tela do navegador.

Ex: Abre a página do dashboard.

2. Front-End

HTML/CSS exibem o visual e JavaScript inicia a requisição HTTP GET para coletar dados.

3. HTTP / API

A requisição viaja pela rede e atinge o endpoint seguro exposto pela API backend.

4. Back-End

O motor C# / .NET processa a segurança, regras e executa a conexão segura de dados.

5. Banco de Dados

Os dados de produtos, clientes ou vendas são armazenados fisicamente e retornados.

Fluxo Integrado em 8 Etapas:

1) HTML renderiza tela -> 2) JS dispara GET /api/vendas -> 3) Requisição viaja -> 4) API recebe e encaminha -> 5) C# processa regras de acesso -> 6) Banco responde com dados brutos -> 7) Backend converte e envia resposta em JSON -> 8) JS recebe JSON e reconstrói o HTML dinamicamente.

PARTE 2

Desenvolvimento Prático e Orientado

Pergunta de Transição: “Hoje nosso projeto possui informações colocadas diretamente no JavaScript. Mas e se tivéssemos 10 mil vendas? Onde esses dados ficariam?”

Foco na Organização do Front-End e Mapeamento Dinâmico de Dados

Os alunos avançam nos componentes visuais enquanto preparam o código para a futura integração

Atividades do Desenvolvimento Prático

1. Refinar e solidificar a estrutura HTML das páginas.
2. Melhorar o design e a consistência visual via CSS.
3. Codificar componentes reutilizáveis e cards de dashboard.
4. Desenvolver interações inteligentes usando JavaScript.
5. Reorganizar diretórios e a hierarquia dos arquivos do projeto.
6. Depurar erros pendentes de console e de exibição de tela.
7. Aplicar técnicas de usabilidade e boas práticas de UX.

Nova Exigência: Mapeamento de API

Identificação Dinâmica: Os alunos devem mapear quais dados exibidos hoje estão fixos e deverão vir do Backend.

Instrução: Adicionar comentários no código Javascript indicando os pontos futuros de carregamento via fetch API.

Exemplo de Código para Prática:
// Futuramente este valor virá do endpoint GET /api/vendas

Paradigma de Evolução do Projeto:

```
HOJE: JavaScript → Dados Estáticos → Renderização HTML
FUTURO: Banco de Dados → C#/.NET → API (JSON) → JavaScript → HTML
```

Próximos Passos: Construindo o Primeiro Endpoint em .NET

A transição suave para o desenvolvimento do servidor trará dinamismo ao projeto front-end

Consolidação Teórica: Na próxima sessão, os alunos usarão seus mapeamentos para planejar as rotas do servidor.

Ambiente de Trabalho: Utilizaremos o VS Code para iniciar o ecossistema C# e .NET de desenvolvimento de APIs.

O Primeiro Passo Prático: Criação de um único endpoint GET inicial que retorne dados estruturados simples no formato JSON.

Integração Gradual: Com o primeiro endpoint funcional em mãos, a turma aprenderá a substituir seus dados fixos no JavaScript por chamadas fetch() reais ao servidor local.